

How to Create Synchronous Method using Task in C#

In this article, I am going to discuss **How to Create Synchronous Method using Task in C#** with Examples. Please read our previous article where we discussed [How to Cancel a Task in C# using Cancellation Token in C#](#) with Examples.

How to Create Synchronous Method using Task in C#?

There will be occasions when you will have a method with an asynchronous signature, that is return Task, but its implementation will be Synchronous. One reason for this is that you have to implement a basic interface that returns a task and the implementation is synchronous.

Another reason could be that in your unit tests you need to mock asynchronous methods. And the implementation is going to be synchronous. To solve these problems, we can use axillary methods like CompletedTask, FromResult, FromException, and FromCanceled.

1. With Task.CompletedTask method, we can implement a method that returns a task, but that is synchronous.
2. With Task.FromResult method, we can implement a method that is task<T>, but that it is synchronous. And of course, we can return a value that will be wrapped inside of a task.
3. With Task.FromException, we can create a task that completed with an error
4. With Task.FromCanceled, we can create a task that has been canceled.

It is important to be able to create a task that has an Exception or that is Cancelled because of your unit tests, you may want to test a method that has to handle a faulted task or a task with an exception, or a task that has been canceled.

Note: These operations are available from .NET Framework 4.6, 4.6.1, 4.6.2, 4.7, 4.7.1, 4.7.2, 4.8, and .NET Core All versions.

Syntax to use Task.CompletedTask in C#:

CompletedTask: This is a property of the Task class. It returns the successfully completed task.

For a better understanding of how to use CompletedTask in C#, please have a look at the below image. Here, the method returns Task but here we have not used the async operator in the method signature. So, when we want to implement a synchronous method that returns a Task, then we need to use Task.CompletedTask.

```
//Method returning Task but it is synchronous
public static Task SomeMethod1()
{
    //Do Some Task

    //When your method returning Task in synchronous, return Task.CompletedTask
    return Task.CompletedTask;
}
```

Syntax to use Task.FromResult in C#:

FromResult<TResult>(TResult result): It Creates a Task that is completed successfully with the specified result. Here, the parameter result specifies the result to store in the completed task. Here, the type parameter TResult specifies the type of the result returned by the task. It returns the successfully completed task.

For a better understanding of how to use FromResult in C#, please have a look at the below image. Here, the method returns Task but here we have not used the async in the method signature. So, when we want to implement a synchronous method that returns a Task<T>, then we need to use Task.FromResult in C#.

```
//Synchronous Method returning Task<T>
public static Task<string> SomeMethod2()
{
    string someValue = "";
    //Do Some Task

    //When your synchronous method returning Task<T>, use Task.FromResult
    return Task.FromResult(someValue);
}
```

Syntax to use Task.FromException in C#:

FromException(Exception exception): It creates a Task that has been completed with a specified exception. Here, the parameter exception specifies the exception with which to complete the task. It returns the faulted task.

For a better understanding of how to use FromException in C#, please have a look at the below image. Here, the synchronous method returns a Task but with an Exception. So, when we want to implement a synchronous method that returns a Task with Exception then we need to use Task.FromException in C#.

```
//Synchronous Method returning Task with Exception
public static Task SomeMethod3()
{
    //Do Some Task

    //When your synchronous method returning Task with Exception use, Task.FromException
    return Task.FromException(new ApplicationException());
}
```

This is useful when you have a unit test and you want to make sure that the method handle task with exceptions.

Syntax to use Task.FromCanceled in C#:

FromCanceled(CancellationToken cancellationToken): It creates a Task that's completed due to cancellation with a specified cancellation token. Here, the parameter cancellationToken specifies the cancellation token with which to complete the task. It returned the canceled task. This method will throw ArgumentOutOfRangeException if the cancellation has not been requested for cancellationToken; it's CancellationToken.IsCancellationRequested property is false.

For a better understanding of how to use FromCanceled in C#, please have a look at the below image. Here, the synchronous method cancels a task. So, when we want to implement a synchronous method that cancels a Task, then we need to use Task.FromCanceled in C#.

```
//Synchronous Method Cancelling a Task
public static Task SomeMethod4()
{
    //Do Some Task
    var CTS = new CancellationTokenSource(5000);
    //When your synchronous method cancelling a Task, Task.FromCanceled
    return Task.FromCanceled(CTS.Token);
}
```

Example to Understand How to Create Synchronous Method using Task in C#:

The following example shows the use of all the above four methods.

```
using System;
using System.Threading;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started");
            SomeMethod1();
            SomeMethod2();
            SomeMethod3();
            SomeMethod4();

            Console.WriteLine("Main Method Completed");
            Console.ReadKey();
        }
        //Method returning Task but it is synchronous
        public static Task SomeMethod1()
        {
            //Do Some Task
            Console.WriteLine("SomeMethod1 Executing, It does not return anything");
            //When your method returning Task in synchronous, return Task.CompletedTask
        }
    }
}
```

```

        return Task.CompletedTask;
    }
    //Synchronous Method returning Task<T>
    public static Task<string> SomeMethod2()
    {
        string someValue = "";
        someValue = "SomeMethod2 Returing a String";
        Console.WriteLine("SomeMethod2 Executing, It return a string");
        //When your synchronous method returning Task<T>, use Task.FromResult
        return Task.FromResult(someValue);
    }
    //Synchronous Method returning Task with Exception
    public static Task SomeMethod3()
    {
        Console.WriteLine("SomeMethod3 Executing, It will throw an Exception");
        //When your synchronous method returning Task with Exception use,
        Task.FromException
        return Task.FromException(new InvalidOperationException());
    }
    //Synchronous Method Cancelling a Task
    public static Task SomeMethod4()
    {
        CancellationTokenSource cts = new CancellationTokenSource();
        cts.Cancel();
        Console.WriteLine("SomeMethod4 Executing, It will Cancel the Task
        Exception");
        //When your synchronous method cancelling a Task, Task.FromCanceled
        return Task.FromCanceled(cts.Token);
    }
}
}

```

Output:

```

Main Method Started
SomeMethod1 Executing, It does not return anything
SomeMethod2 Executing, It return a string
SomeMethod3 Executing, It will throw an Exception
SomeMethod4 Executing, It will Cancel the Task Exception
Main Method Completed

```

In the next article, I am going to discuss the [Retry Pattern of Asynchronous Programming in C#](#) with Examples. Here, in this article, I try to explain **How to Create Synchronous Method using Task in C#** with Examples. I hope you enjoy this How to Create Synchronous Method using Task in C# with Examples article.